

Kryptografie – Projekt 2 – Project CERBERUS (xkucik00)

Cieľom projektu je vytvoriť jednoduché Boneh-Lynn-Shacham (BLS) schéma pre multi-party podpisy využívajúci kryptografiu na eliptických krivkách, z dôvodu efektívnosti lineárnych operácií. Na tomto základe bude pracovať nami vytvorený systém CERBERUS.

Na to, aby sme mohli vytvoriť multi-party podpisy bolo ako prvé potrebné vytvoriť generáciu kľúčov. To sme vytvorili na základe eliptických krivkách, kde sme zo súkromného kľúča (sk) a generátora (G) vytvorili verejný kľúč: $PK = sk \cdot G$. V rámci implementácií sme mali za úlohu zvoliť bezpečný generátor náhodných čísel pre vytvorenie súkromného kľúča sk . K tomu sme využili knižnicu *secrets* a veľkosť kľúča stanovili na 256 bitov, keďže sa v projekte využíva sha256 ako hash funkcia. Napokon pomocou funkcie *multiply* sme dostali PK . Ďalšia potrebná funkcia bola na elektronický podpis, na základe rovnice: $S = sk \cdot H(m)$, kde S je výsledný elektronický podpis.

Podstatná vec pre vytvorenie BLS systému bola potreba agregácie kľúčov, kde sa jedná o vytvorenie spoločného verejného kľúča a podpisu, ktorý pozostáva z sumy PK a sumy S . Tieto kľúče slúžia na verifikáciu toho, že celá skupina schválila danú informáciu. Posledná implementovaná funkcia v základe BLS bola potreba verifikovať tieto agregované kľúče na základe rovnice $e(S_{agg}, G_2) = e(H(m), PK_{agg})$, kde S_{agg} je agregovaný podpis vzniknutý súčtom podpisov každého administrátora, G_2 je bod na eliptickej krivke, e párovacia funkcia, ktorá slúži na verifikáciu, $H(m)$, je hash správy a PK_{agg} je agregovaný verejný kľúč vytvorený rovnako ako agregovaný podpis, avšak sumou verejných kľúčov administrátorov. Implementačne sa postupovalo podľa daných rovníc a poskytnutých funkcií *add* pre sumu pri agregácii a *pairing* pri verifikácii.

Pri radosti z vytvoreného bezpečného BLS systému prichádza Bob, ktorý vyvráti tvrdenie o bezpečnosti. Náš systém umožňuje vytvorenie darebáckeho PK na základe znalosti kľúčov ostatných administrátorov Alice (A) a Charlie (C). Prakticky mu stačí iba jeho súkromný kľúč a verejný kľúč administrátorov. Zo svojho súkromného kľúča si vytvorí verejný, ktorý avšak chce utajiť. Darebácky kľúč vytvorí na základe sumy svojho kľúča a negácie kľúčov A a C, nasledovne podľa rovnice: $PK_{rogue} = (sk_B \cdot G_2) - PK_A - PK_C$. Takto vytvorením kľúčom vie Bob vytvoriť podpis, ktorý sa overí ako platný podpis celej skupiny, hoci A ani C nič nepodpísali.

Z tohto dôvodu sme zaviedli bezpečnostné opatrenie *Proof of Possession*. Jedná sa o to, že pre vytvorenie podpisu budeme vyžadovať sa overiť do systému na základe sk a hashu PK . Tu teda Bobovi nepomôže darebácky kľúč, lebo nie je schopný sa dostať cez verifikáciu do systému, lebo na to sa potrebuje registrovať pomocou PK a disponujúcim sk . Keďže Bob nepozná súkromné kľúče A a C, nedokáže vytvoriť dôkaz vlastníctva pre takto agregovaný kľúč. Pri implementácii pre serializáciu PK bola využitá funkcia *pickle*, ktorá serializovala PK pre hash funkciu.

V rámci riešenia projektu je priložená aj základná sada testov (*test_suite.py*) na ktorej sa testovala funkcionálnosť projektu.